

AI Night Guardian

Acoustic detection of night-time incidents in care homes,
with on-device inference and no audio retention

Jan Díaz Carreras-Candi Guillem Guilera Aulet Joan Montobbio Palmer

ETSETB — Universitat Politècnica de Catalunya
HackEstiu 2026

Abstract

In care homes, night staff check each room on rounds roughly once an hour, which leaves up to an hour in which nobody knows what is happening inside a room. AI Night Guardian is a low-cost device that listens to a room and alerts staff the moment it detects an incident, running a neural network locally on an Arduino UNO Q and without ever storing or transmitting audio. This document describes the system, the dataset pipeline, the model, the on-device decision logic and the mobile application, along with the measured results and the known limitations.

Contents

1	Problem	3
2	System overview	3
2.1	Architecture	3
2.2	Privacy by design	4
3	Hardware	4
4	Repository layout	5
5	Dataset	6
5.1	Cleaning	6
5.2	From 47 source labels to 5 classes	6
5.3	Pipeline	7
6	Model	7
6.1	Class design	7
6.2	Label quality	8
6.3	Architecture	8
6.4	The silence class	9
6.5	Signal level and gain augmentation	9
6.6	Results	10
7	On-device runtime	10
7.1	Decision logic	10
7.2	Listening switch	12
7.3	Telemetry robustness	12

8	Mobile application	12
8.1	Communication protocols	13
9	Reproducing the system	13
9.1	Rebuilding the dataset	13
9.2	Running the device	13
9.3	Calibrating for a new room	14
10	Limitations	14
11	Future work	15
	Resources	15

1 Problem

In care homes, night staff work by rounds: they walk into each room to check that the resident is all right. Depending on the facility, a given room gets a visit roughly once an hour.

That means there is up to an hour in which nobody knows what is happening inside a room. If someone falls, if someone is in distress, it is noticed on the next round — not when it happens.

The obvious technical fix is a camera or an always-on microphone. But these are people’s bedrooms, and continuous recording in a resident’s room is not acceptable, both as a matter of dignity and under GDPR. Any system deployed there has to work without keeping the audio.

Goal. A device that detects incidents at the moment they occur, costs little enough to be deployed in every room, and processes everything locally so that no audio ever leaves the room.

2 System overview

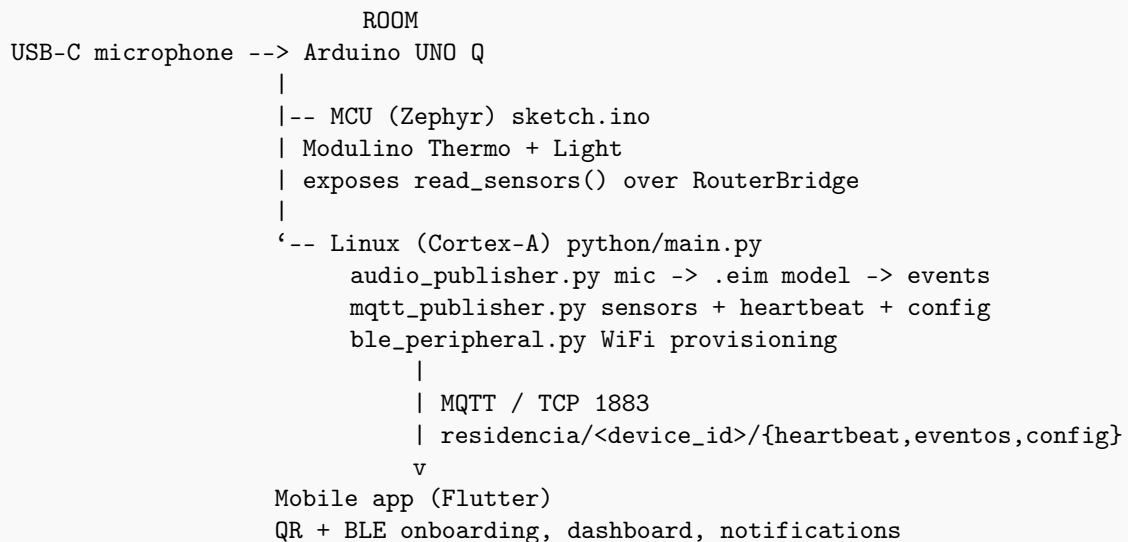
The device classifies the sound of the room into five categories, three of which raise an alert. When one fires, it publishes a short JSON message over MQTT and the staff’s mobile app raises a notification.

Class	Covers	Action
impacte	Impacts, falls, doors	Alert
veu_angoixa	Screaming, crying, distress	Alert
tos	Coughing fits	Alert
normal	Speech, snoring, rain, traffic, alarms	None
silenci	A quiet room	None

Table 1: Classes and the action each one triggers on the device. The class names are the labels of the deployed impulse; the preprocessing scripts use the English equivalents (**impact**, **distressed_voice**, **cough**, **normal**).

Two non-acoustic alerts complete the system: temperature outside a configurable range, which matters during heatwaves, and changes in ambient light, which can flag a resident getting up in the middle of the night.

2.1 Architecture



The split matters: the microcontroller only reads the two I²C sensors and answers requests over RouterBridge, while everything acoustic happens on the Linux side, where the Edge Impulse™ runtime and the MQTT client live. Audio never reaches the MCU.

Incident flow. Microphone → 1 s window in RAM → Mel-filterbank energies → **audio window discarded** → inference → if the energy gate, the confidence threshold, the burst rule and the refractory period all pass, a JSON text message is published over MQTT.

2.2 Privacy by design

This is the core constraint of the project and the reason it is deployable in a care home.

- No personal or biometric data is processed or stored.
- The acoustic signal is captured straight into memory.
- Once inference has run, the audio window is immediately and irreversibly discarded, having never been written anywhere.
- No audio is transmitted outside the room.
- The only thing that leaves the device is text metadata. The app’s local incident history stores the same metadata and nothing else.

```
{
  "type": "event",
  "device_id": "AING-9D67B1",
  "ts": "2026-09-12T02:40:11+0200",
  "payload": {
    "room": "Room 14",
    "event_type": "impacte",
    "confidence": 0.85
  }
}
```

Legally, the system behaves as an automated emergency sensor rather than a listening microphone. The app additionally exposes a per-room listening switch; turning it off does not merely discard results, it shuts down the model process and releases the audio device (Section 7.2).

3 Hardware

Component	Role
Arduino UNO Q (4 GB)	Main board; Linux (Cortex-A) plus a Zephyr MCU
T’nB Influence Lapel Mic USB-C	Audio capture, 16 kHz mono
USB-C hub	Connects microphone and power to the board
Modulino Thermo	Room temperature and humidity
Modulino Light	Ambient light level
3D printed enclosure	Four parts; STL and OBJ in the repository

Table 2: Bill of materials.

The two Modulinos are daisy-chained on the board’s Qwiic I²C connector and read from the MCU through the `Arduino_Modulino` library; no extra wiring is needed beyond the chain. The microphone is a plain USB audio device: it is enumerated by Linux, not by the MCU, and the inference process opens it by index (`AUDIO_DEVICE_ID`).

Enclosure. Four printed parts, in `ARDUINO/3D_DESIGNS/` as both STL and OBJ: `case_front`, `case_back`, `microphone_support` and `modulinos_support`. The two supports hold the microphone and the sensor chain in place so that the acoustic port and the light sensor are not obstructed by the cabling.

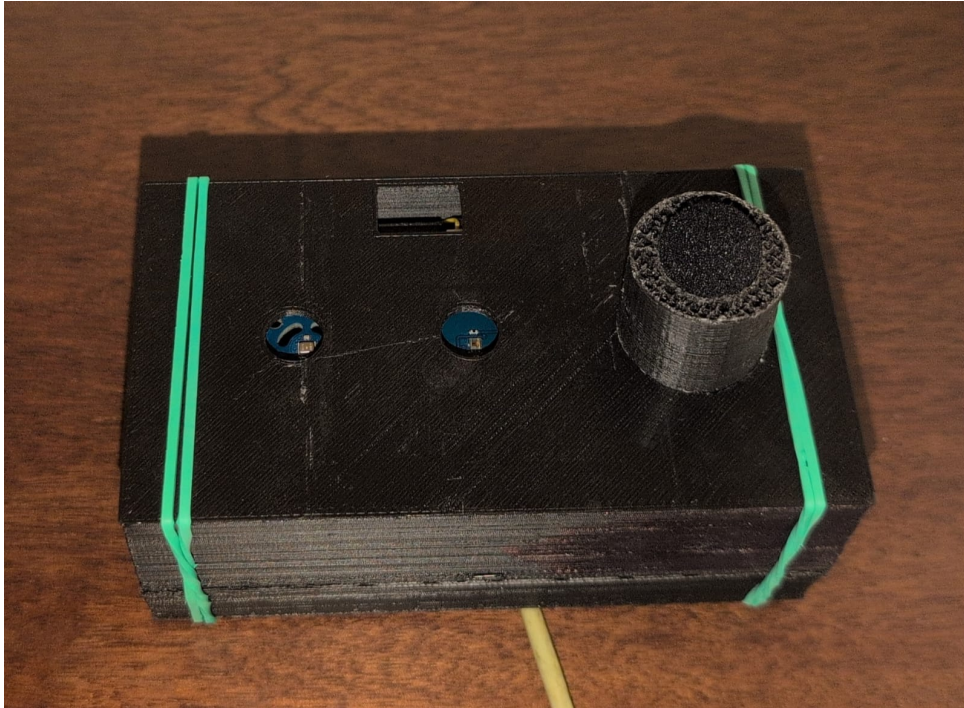


Figure 1: Device assembled.

4 Repository layout

Everything needed to rebuild the system is in the repository, including the audio collection and the deployed model binary.

```
AI-Night-Guardian/
|-- APP/ Flutter application (Android / iOS / desktop)
| |-- lib/data/ MQTT service, notifications, event log, storage
| |-- lib/presentation/ Screens: splash, home, devices, detail, QR,
| | BLE scan, WiFi provisioning, dashboard, settings
| '-- assets/translations/ ca.json, es.json, en.json
|
|-- ARDUINO/
| |-- 3D_DESIGNS/ Enclosure: 4 parts, STL + OBJ
| '-- ai_night_guardian_firmware.tar.gz
| sketch/sketch.ino MCU: Modulino + RouterBridge
| python/main.py Launches the three Linux processes
| ai_night_guardian/
% audio_publisher.py Inference loop and decision logic
% mqtt_publisher.py Sensor thread, heartbeat, config
% ble_peripheral.py GATT WiFi provisioning
% device_identity.py device_id, provisioning key, QR
% event_simulator.py Publishes fake incidents for demos
% *.eim Deployed Edge Impulse model (14 MB)
|
|-- DATA PREPROCESSING/ Dataset cleaning and preparation (6 scripts)
| '-- recordings/ 501 two-second clips of the target room
|
|-- DATASET/
| |-- creation/Scripts/ Extractors for the four source datasets
| '-- final_dataset_16khz/ 30,487 WAV clips, 16 kHz mono
|
```

```
|-- DOCUMENTATION/ This document and the design notes
'-- LICENSE MIT
```

(The lines marked with % above are the contents of the firmware archive, which is shipped compressed.)

5 Dataset

The training set is built from public audio: 30 487 clips normalised to 16 kHz mono, in `DATASET/final_dataset_16`

Source	Clips	Origin
AudioSet (Google)	10 463	10 s segments from YouTube
FSD50K (UPF)	13 642	Zenodo
COUGHVID	5 902	Zenodo — verified coughing
ESC-50	480	GitHub
Total	30 487	

Table 3: Source datasets. One extractor script per source, in `DATASET/creation/Scripts/`.

Each source needs a different extraction strategy. AudioSet ships no audio at all: the CSVs list YouTube IDs and time offsets, so clips are fetched with `yt-dlp` and cut with FFmpeg, and some are permanently unavailable. FSD50K requires crossing numeric file names against its ground-truth CSV. ESC-50 is filtered on the numeric `target` column of `esc50.csv`. COUGHVID is filtered on its own quality metadata, keeping only clips with `cough_detected` ≥ 0.95 . A final `UNIFICADOR.py` pass re-encodes everything to 16 kHz mono.

Why 16 kHz mono. 16 kHz is the TinyML standard: by Nyquist it captures up to 8 kHz, and human voice, snoring and impacts carry practically all of their acoustic information below 4 kHz. Higher rates would only add useless high frequencies and inflate memory use during spectrogram generation. Mono, because the model does not need to know which side a sound came from. Re-encoding is lossless in practice — the files are uncompressed PCM — and it rewrites the WAV headers, which removes the stray metadata that otherwise causes upload errors in Edge Impulse™.

5.1 Cleaning

The raw collection is not usable as-is. Automated inspection found:

Exact duplicates (MD5)	2 634
Corrupt files (0.0 s)	2 621
Clips shorter than 1 s	1 540

Table 4: Problems found in the raw collection. Almost 17% was unusable.

Duplicates matter beyond wasted space: with several sources downloaded separately, the same file appears under different names, and if it is not removed before the train/test split it leaks across it.

5.2 From 47 source labels to 5 classes

The source labels are the ontology tags of the original datasets — 47 of them across the four sources. `build_manifest.py` maps each one to a final class, and deliberately *discards* some:

Discarded label	Reason
laughter, laughing	Shares vocal mechanism and burst structure with distress; rare at night
sneeze, burping	Not incidents, and they blur the cough boundary
respiratorysounds	Parent category of cough, snoring and breathing in AudioSet: contaminates both sides of the cough/normal boundary
insidesmallroom	Room-tone tag, not an event
silence	Handled by the energy gate on the device, and later as its own recorded class

Table 5: Labels excluded from training and why.

Each class is then capped, with sampling stratified by original label so that a class is not filled by a single source (for instance **normal** consisting only of speech). Caps: 2000 for **impact** and **veu_angoixa**, 1500 for **tos**, 2500 for **normal**, 800 for anything else; the critical classes stay below their cap in practice. The residual imbalance is handled with auto-weighted classes during training. All sampling uses a fixed seed (42), so the manifest is reproducible.

5.3 Pipeline

The scripts in `DATA PREPROCESSING/` turn the raw collection into a training set. They run in this order:

1. `build_manifest.py` — consolidates the source labels, detects duplicates by MD5, flags corrupt and short files, applies the caps, and writes one CSV row per file with a `keep_for_training` decision.
2. `materialize_dataset.py` — copies the selected clips into one folder per class.
3. `segment_events.py` — energy-based segmentation into 2s samples: it finds every region more than 10 dB above the clip’s own noise floor, merges regions closer than 0.25 s, drops peaks shorter than 0.08 s and exports up to 3 per clip. Continuous classes (**normal**) get random windows instead, because looking for peaks there would manufacture transients that resemble impacts.
4. `yamnet_clean.py` — semantic segmentation, which supersedes the previous step for the final dataset (Section 6.2).
5. `augment_gain.py` — random gain per clip (Section 6.5).
6. `prepare_room_audio.py` — builds the **silenci** class from a recording of the target room, as 2 s windows with 1 s hop.

Planned versus implemented augmentation. The initial design notes proposed background-noise injection, pitch shifting, time stretching and SpecAugment in addition to gain. Only gain augmentation was implemented, because it was the transformation that addressed a failure actually observed on the device (Section 6.5); the others remain as future work, and Edge Impulse™ can apply noise and time-shift augmentation in the training block itself.

6 Model

6.1 Class design

The first version had ten classes and scored 46.1%. The problem was not acoustic difficulty but design: most of those classes existed only to say “this is not an incident”, and they competed over

distinctions that change nothing about what the system does.

The final taxonomy has five classes, one per action. Three of the merges are worth explaining.

door belongs with impacte. A door slamming and a body hitting the floor produce the same low-frequency transient. Trying to separate them left the fall class at 9% accuracy; merging them took it to 74%. At night, inside a room, a door is also worth checking.

Screams and crying are merged. They confused each other (26.7%) and raise exactly the same alert, so separating them asked the model for a distinction that serves no purpose.

Snoring goes to normal, and is not discarded. It is the dominant sound all night in a care home. Without somewhere correct to put it, the model files it as coughing or distress and raises false alarms every night.

The granularity given up here is recovered on the device: the mapping from class to action is deterministic and lives in the code, not in the network.

6.2 Label quality

For several iterations the model sat around 50–53% and nothing moved it: neither more epochs, nor more capacity, nor a different architecture.

The clue was that training and validation accuracy were nearly identical. A model that cannot fit its own training data does not have a capacity problem; it has a data problem.

FSD50K and AudioSet use *weak labels*: the tag means the sound appears somewhere in a ten-second clip, not that it fills it. A clip labelled **cough** may hold one second of coughing and nine of something else. Energy-based segmentation does not fix this — it takes the loudest part of the clip, which is not necessarily the labelled sound.

To measure it, the collection was passed through YAMNet — a model pretrained on AudioSet that classifies 521 sound types every 0.48 s — keeping only 2 s fragments where it confirms the expected sound with confidence at least 0.10, at most three per clip. Clips where YAMNet confirms nothing are dropped entirely, which is precisely the case where the label was probably wrong.

Class	Clips discarded
veu_angoixa	31%
impacte	27%
normal	8%
tos	7%

Table 6: Fraction of clips where YAMNet confirmed nothing. The two critical classes had the dirtiest labels; **tos**, always the best performer, the cleanest.

With the filtered set, accuracy went from 64.4% to 75.5% *while training on less data*. A more aggressive filter (confidence 0.20 instead of 0.10) made things worse: it removed hard but legitimate examples, which are the ones needed to generalise. **normal** keeps only one fragment per clip rather than three, because it starts with far more clips and loses very few of them to the filter, which would otherwise leave it with nearly four times the samples of **veu_angoixa**.

6.3 Architecture

- **Preprocessing:** Mel-filterbank energies (MFE), 1 s window with 500 ms stride, 40 filters, 256-point FFT, noise floor -102 dB.

- **Network:** MobileNetV2 0.35, pretrained, Edge Impulse™ keyword-spotting transfer learning block, auto-weighted classes.
- **Deployment:** `.eim` binary for Linux aarch64, run by the Edge Impulse™ Python runtime on the UNO Q — 10 ms latency, 177 kB of RAM. The exact file shipped in the repository is `granny-guardians-fr-linux-aarch64-v8-impulse-#1.eim`.

Transfer learning was compared against a 1D CNN trained from scratch on identical data and configuration:

Class	Transfer learning	From scratch
<code>impacte</code>	82.8%	65.1%
<code>veu_angoixa</code>	69.0%	36.4%
<code>tos</code>	81.6%	85.5%
Overall	75.0%	68.3%

Table 7: Pretrained weights are worth about 8 points overall and more than 25 on the smallest class. Giving the from-scratch network more capacity (32/64 filters instead of 8/16) made it worse, not better: 67.1%.

6.4 The silence class

Once deployed, the device raised `impacte` alerts at 87–91% confidence in an empty room.

Every step of the pipeline keeps only fragments with audible sound, so the model had never seen silence — and at night silence is most of the input. Handed something unlike anything in its training, the classifier had to spread the probability across the classes it knew. It had no option for “nothing is happening”.

Recording the target room with the deployment microphone and adding it as its own class fixed it: it learns at 97.9%, the most separable class in the set. The 501 two-second clips used are in `DATA PREPROCESSING/recordings/`. Silence is *not* passed through gain augmentation, since it is a class defined precisely by its level.

6.5 Signal level and gain augmentation

	RMS (int16)	dBFS
Training clip	4 857	−16.6
Clap near the microphone	741	−33
Clap across the room	365	−39
Fall off a sofa	148	−47
Normal conversation	50–70	−54
Quiet room	24	−62

Table 8: Measured signal levels. The deployed system works about 30 dB below the training data.

The MFE block is not level-invariant: it normalises against a fixed noise floor, so the same sound at two volumes produces different features. Reproducing the computation outside Edge Impulse™ with the project’s parameters confirmed it — features shift by 0.33 on a 0–1 scale, while the spectral shape is preserved almost exactly (correlation 0.96).

There is no single level to target either: the same event reads 148 or 741 depending on distance to the microphone. So rather than matching a level, the model is trained with a random gain per clip targeting between −50 and −15 dBFS, forcing it to rely on spectral shape instead of absolute volume. 15% of clips are left at their original level as a clean reference, gains are clamped to

avoid clipping, and the output is written as 16-bit PCM on purpose: the real microphone is also 16-bit, so the loss of resolution at low levels is part of what the model has to cope with. This is also what makes the model robust to *distance*, which is the variable that changes most from room to room.

6.6 Results

Iteration	Change	Accuracy
1	10 classes, own CNN	46.1%
2	8 classes	53.3%
3	4 classes, transfer learning	60.7%
4	More data on critical classes	64.4%
5	YAMNet label cleaning	75.5%
6	silenci class	76.8%
7	Gain augmentation	75.0%

Table 9: Validation accuracy across iterations.

Final model, validation set: **silenci** 97.9%, **impacte** 82.5%, **tos** 78.9%, **veu_angoixa** 69.0%, **normal** 62.4%. AUC-ROC 0.94; weighted precision, recall and F1 all 0.75. Per-class F1 follows the same order as the thresholds chosen on the device: **tos** 0.81, **impacte** 0.79, **veu_angoixa** 0.66.

On the test figure. Edge Impulse™’s Model testing applies a single 0.6 confidence threshold and files everything below it as UNCERTAIN, which accounts for 27–35% of windows. Those windows are mostly classified correctly; they simply do not clear the cut-off. The device uses per-class thresholds instead, so a single global threshold measures a configuration that exists nowhere. The threshold-independent figures — AUC-ROC 0.94 and F1 0.75 — are the meaningful ones.

A note on iteration 7. Gain augmentation lowers the score and improves the system. On the real device the previous version raised occasional impact alerts during ordinary conversation; this one does not. The validation set cannot see the difference, because the problem it solves — sensitivity to level and distance — only appears in deployment. Validation accuracy is not the product metric.

7 On-device runtime

Three processes run on the Linux side, started by `python/main.py`: the inference loop, the sensor and telemetry publisher, and the BLE provisioning peripheral.

Priorities. Inference is the dominant load and does not yield the CPU voluntarily, which was starving the `read_sensors` requests to the MCU — temperature and light arrived as `null` in the heartbeat. The audio process is therefore launched with `nice -n 10` and the sensor reader runs in its own thread at a raised priority when permitted. The sensor thread spends nearly all its time blocked waiting for the MCU, so this costs the classifier no measurable real-time margin.

7.1 Decision logic

The impulse window is 1 s with a 500 ms stride, but the Linux runner feeds it continuously in chunks of about 64 ms, so roughly 16 overlapping classifications per second reach the decision

logic — over an eight-hour night, hundreds of thousands per room. Even a tiny false-positive rate becomes an unusable number of notifications, and a system that cries wolf gets muted in the first week. Four filters sit between the model and the phone.

Energy gate. If the RMS of the window is below a calibrated floor, the classification is discarded before it is even looked at. The RMS is computed over the same ≈ 1 s the model sees, accumulated from the 64 ms chunks, because a single chunk can land in a pause or right on a click and give erratic values.

Per-class confidence thresholds. The model is less confident on distress than on impacts, so a single global threshold either misses screams or floods on impacts. In a safety system a false alarm costs less than a missed event, so the critical classes get lower thresholds.

Cough burst rule. A single cough, or clearing one’s throat, is not an incident; a persistent fit or a choking episode is. Coughs are counted within a sliding window and an alert is raised only once enough *distinct* coughs accumulate — consecutive overlapping windows of the same cough do not count twice.

Per-class refractory period. After an alert for a class, further alerts for that class are suppressed for a configurable window. One notification per coughing fit, not eighty. The timestamp is kept per class, not globally: with a single timestamp the sequence impact $\rightarrow$ cough $\rightarrow$ impact let the second impact through immediately.

Parameter	Meaning	Value
SILENCE_RMS	Energy gate, int16 RMS over 1 s	80
CONFIDENCE_THRESHOLDS	veu_angoixa	0.65
	impacte	0.75
	tos	0.65
DEFAULT_THRESHOLD	Any other class	0.85
DEBOUNCE_SECONDS	Refractory period, per class	60 s
COUGH_MIN_COUNT	Coughs needed to alert	6
COUGH_WINDOW_SECONDS	Sliding window for the count	60 s
COUGH_MIN_GAP_SECONDS	Minimum gap between two coughs	0.8 s

Table 10: Decision parameters, at the top of `audio_publisher.py`.

The energy gate is derived from the room recording rather than guessed: the background measured 0.00074 RMS median (-62.6 dBFS, ≈ 24 in int16) and 0.00121 at the 95th percentile (-58.4 dBFS, ≈ 40), and the threshold is set at twice the 95th percentile.

```

rms = _rms(np.concatenate(audio_buffer))
if rms < SILENCE_RMS:
    continue

top_label = max(scores, key=scores.get)
if top_label in IGNORED_LABELS: # normal, silenci
    continue
if scores[top_label] < CONFIDENCE_THRESHOLDS.get(top_label, DEFAULT_THRESHOLD):
    continue

now = time.time()
if top_label == COUGH_LABEL and not _is_cough_fit(now):
    continue

```

```
if now - last_event_ts.get(top_label, 0.0) < DEBOUNCE_SECONDS:
    continue
last_event_ts[top_label] = now
_publish_event(top_label, scores[top_label])
```

7.2 Listening switch

The switch in the app writes a flag that the device polls at most once a second. Turning it off raises an exception that unwinds the runner’s context manager, which terminates the model process and releases the audio device; the loop then idles until the flag comes back. Merely ignoring the results would save neither CPU nor temperature, and would leave the microphone open.

Two details are deliberate. Any read error on the flag file is treated as “keep listening”: in a safety system the fail-safe state is to keep watching. And the whole loop retries indefinitely, so the process can be started before the microphone is plugged in or the model copied, and picks up on its own once both exist.

7.3 Telemetry robustness

`Bridge.call()` is blocking with a 10s timeout, so reading the sensors from the publishing loop would stall the heartbeat whenever the MCU hiccups. The reader therefore runs in its own thread and writes into a cache; the publisher only ever reads the cache. Readings older than 60s are published as `null` rather than as a stale value — an honest gap is better than a wrong number — and consecutive failures are logged on the first and then every tenth, so a downed MCU does not flood the log. MQTT reconnection is retried on a 10s backoff, and the heartbeat keeps its cadence throughout.

8 Mobile application

A Flutter application, built for Android (and buildable for iOS, Linux, macOS, Windows and web), in three languages: Catalan, Spanish and English.

Onboarding. Splash → *Add device* → scan the **QR** shown by the device, which carries its `device_id` and provisioning key → **BLE scan** → **WiFi** form, with the network list read from a scan and a room name → the device connects and publishes its first heartbeat → the room appears on the dashboard. Required permissions: Bluetooth, camera, location (needed for BLE scanning on Android) and notifications.

Running screens. A home screen with one live card per room; a device detail screen with temperature, light, uptime, signal, last incident and the listening switch, plus a technical dialog showing the MQTT topic, the age of the last heartbeat and of the last sensor reading, and the broker state; a device list with swipe actions to rename or unpair; and settings for language and notifications.

Two kinds of notification. Incidents are delivered at maximum importance. Loss of signal is delivered at normal importance and is a safety feature in its own right: a watchdog checks every 30s and, if no heartbeat has arrived for 90s — eighteen missed heartbeats — the room is marked offline and the staff are told, once, that the sensor stopped reporting. A device that has silently died must not look like a quiet room. Sensor values that stop updating are flagged as stale in the interface rather than shown as current.

Local history. Incidents are kept on the phone for 24 hours, at most 30 per device, as metadata only — room, type, confidence, timestamp. No audio exists to store.

8.1 Communication protocols

All MQTT messages are JSON with `v`, `type`, `device_id`, `ts` and `payload`.

Topic	Direction	Contents
<code>residencia/<id>/heartbeat</code>	device → app	Liveness every 5 s
<code>residencia/<id>/eventos</code>	device → app	Detected incident (QoS 1)
<code>residencia/<id>/config</code>	app → device	Configuration (QoS 1)

Table 11: MQTT topics.

The heartbeat carries the room name, firmware version, uptime, signal strength, temperature, light level and the current state of the listening switch, so the app can show the room as online without any polling. The configuration topic currently carries one key, `{"listening": true|false}`.

WiFi provisioning uses a Nordic UART style BLE service (`6e400001-b5a3-f393-e0a9-e50e24dcca9e`): the app writes SSID, password and room name to the TX characteristic, and subscribes to a STATUS characteristic that reports `connecting`, `connected` with the assigned IP, or `failed` with the reason. The device brings the network up with `nmcli`.

9 Reproducing the system

Everything is open source under the MIT licence. The audio collection, the preprocessing scripts and the deployed model binary are all in the repository, so any stage can be reproduced without rebuilding the previous one.

9.1 Rebuilding the dataset

```
pip install numpy soundfile
pip install tensorflow tensorflow-hub # only for yamnet_clean.py

cd "DATA PREPROCESSING"
python build_manifest.py ../DATASET/final_dataset_16khz manifest.csv
python materialize_dataset.py manifest.csv dataset_ready
python yamnet_clean.py dataset_ready dataset_ready_yamnet
python augment_gain.py dataset_ready_yamnet dataset_ready_gain
python prepare_room_audio.py recording.wav dataset_ready_gain/silenci
```

The result is one folder per class of 2-second WAV clips at 16 kHz mono, ready to upload to Edge Impulse™. To re-extract the collection from the original sources instead, the four extractors and the unifier are in `DATASET/creation/Scripts/`. Alternatively, the Edge Impulse™ project is public and can be cloned directly.

9.2 Running the device

Requirements: Arduino CLI with the `arduino:zephyr` platform, Python 3.11+ on the board's Linux side, `nmcli` (NetworkManager) and BlueZ.

```
tar xzf ARDUINO/ai_night_guardian_firmware.tar.gz
cd ai_night_guardian_firmware
pip install -r python/requirements.txt # qrcode, paho-mqtt, edge-impulse-linux
```

```

pip install numpy bluezero

# 1) Build and upload the sketch to the MCU (Modulino + RouterBridge)
cd sketch
arduino-cli compile --profile default . \
  && arduino-cli upload -p <PORT> --profile default .

# 2) Generate the device identity and its provisioning QR
cd ../ai_night_guardian
python3 device_identity.py

# 3) Everything at once: BLE + telemetry + inference
cd ..
python3 python/main.py

```

The processes can also be run individually for debugging: `ble_peripheral.py` (needs root), `mqtt_publisher.py` and `audio_publisher.py`. `event_simulator.py` publishes a fake incident, which is the quickest way to test the app end to end without making noise. Set `AUDIO_DEVICE_ID` in `audio_publisher.py` to the index of the USB microphone as Linux enumerates it.

9.3 Calibrating for a new room

Gain augmentation is what allows the same model to work across rooms without retraining. For best results there is an optional calibration step: record roughly fifteen minutes of the target room with no incidents — mostly empty and quiet, with a few minutes of television and of ordinary background — and run `prepare_room_audio.py`. It reports the RMS levels, which set the energy gate, and produces clips that can be added as the `silenci` class for a room-specific retrain.

The whole procedure takes minutes and requires no machine-learning knowledge, which is the point: the system works out of the box and calibrates like any other sensor.

10 Limitations

A fall and a closing door sound the same. This is a limitation of the signal, not of the model: the information needed to separate them is not in the audio. Hence the class name `impacte` and the wording “possible incident” rather than “fall detected”. Thunder has the same problem; the light sensor already on the board could help rule it out.

Plosives look like small impacts. The `/p/`, `/t/` and `/k/` sounds in speech are broadband bursts with a sharp attack, and are the main source of false alarms while someone is talking in the room.

Precision is the weak point. Roughly 30% of `normal` windows are classified as some alert class. The decision logic described above is what makes the system usable in spite of this.

No clip in the dataset comes from a real care home. The system is trained on generic public audio and validated in private rooms. A real deployment would need recordings from the actual environment, with the consent that implies.

The MQTT broker is public. The prototype uses `broker.hivemq.com`, which is fine for development and unacceptable for a real deployment: the topics are readable by anyone who knows the device id, and the configuration topic is writable. A production system needs a private broker with TLS and authentication.

Timestamps depend on the network. There is no real-time clock, so the board takes the time from the network; without connectivity, event times drift.

11 Future work

- Improved 3D enclosure: Develop a better-designed 3D-printed enclosure to improve thermal isolation of the Arduino, enhance microphone sound capture, and provide better cable management.
- Larger and more diverse dataset: Expand the training dataset by collecting data from different care-home environments to improve the model's generalization and detection performance.
- Audio feature in the application: Consider integrating an audio component into the application so that users can review detected sounds and determine whether an alert is a false alarm. These user validations could also be used as additional data for further AI model training.
- Improved connectivity and security: Increase connection stability and deploy a private MQTT broker with TLS encryption and authentication to improve the reliability and security of communication between devices.
- Real-world care-home recordings: Collect audio recordings in real care-home environments, with appropriate consent and privacy safeguards, to reduce the domain gap between the training data and real-world deployment conditions.

Resources

- Source code: <https://github.com/Russian03/AI-Night-Guardian>
- Edge Impulse™ project (public): <https://studio.edgeimpulse.com/public/1106722/live>
- Demo video: https://youtu.be/zw-_enrB-u0

The source datasets (AudioSet, FSD50K, COUGHVID, ESC-50) keep their own licences; check them before redistributing the audio collection.